CM Admin Tools

Functionality and step-by-step guide · 10 November 2026

Tools for supporting Alterian CM across the **Dev**, **Test** and **Prod** environments:

Tool	What it is	Use it for
DocLockAdmin	Local website (`http://localhost:5080`)	Document locks, enabling / disabling users, password resets
Remove-AppLogins-Interactive.ps1	PowerShell script	Disabling all `NAME_*` logins in AlterianAuth from the command line
Invoke-QueryAllServers.ps1	PowerShell script	Running a read-only query on every server (and optionally every client database)

All three read the list of servers from `Servers.csv` in this folder.

Contents

1. Before you start
2. Servers.csv
3. DocLockAdmin website
    - Setup
    - Starting and stopping
    - Choosing an environment and database
    - Document locks
    - User logins: searching
    - Application users tab (AlterianAuth logins)
    - Alterian Auth tab (AL.UserIndex)
    - Reset password
    - Audit log
    - Safety checks
    - Settings reference
    - Troubleshooting
4. Remove-AppLogins-Interactive.ps1
5. Invoke-QueryAllServers.ps1
6. What each action changes in SQL
7. Files that must never be committed

---

1. Before you start

You need:

- **Windows** with network access to the SQL servers in `Servers.csv`.
- **.NET 9 SDK** (for the website). Check with `dotnet --list-sdks`.
- **dbatools** PowerShell module (for the scripts): `Install-Module dbatools -Scope CurrentUser`.
- The password of the service account for each environment:

Environment	Windows account
Dev	`IUSA\nuclog`
Test	`ISOTEST\nuclog`
Prod	`ISODOM\nuclog`

The SQL servers use Windows authentication. Your own login may be in a domain the server does not trust, so the website connects **as the environment's account** (the same as `runas /netonly`). You never need a SQL login.

2. Servers.csv

One row per SQL server:

```
Servers,Environment
VIRMZPIMSD125W.INTRA.INFOUSA.COM,Dev
...
```

- The server column may be called `Server` or `Servers`.
- `Environment` must be `Dev`, `Test` or `Prod`.
- Add or remove a server here; restart the website to pick it up.

---

3. DocLockAdmin website

One website for all three environments. You switch environment in the sidebar; each environment connects with its own Windows account.

3.1 Setup

1. Open `DocLockAdmin\appsettings.Local.json` (create it if missing). **This file is excluded from git. Never commit or share it.**
2. Put in the account passwords and the temporary reset password:

```
{
    "DocLock": {
    "ResetPassword": "<temporary password users get after a reset>",
    "Passwords": {
     "Dev":  "<password for IUSA\\nuclog>",
     "Test": "<password for ISOTEST\\nuclog>",
     "Prod": "<password for ISODOM\\nuclog>"
    }
    }
}
```

If a password is not in this file, the site looks in **Windows Credential Manager** for a generic credential named `DocLockAdmin:<Environment>`, and if that is missing too, it asks for it in the console window at startup (and saves it to Credential Manager).

To save one by hand instead:

```
cmdkey /generic:DocLockAdmin:Prod /user:ISODOM\nuclog /pass
```

3. The accounts per environment are set in `DocLockAdmin\appsettings.json` under `Accounts`. You normally don't need to change them.

> Plan: the passwords will move to AWS Secrets Manager. Until then, keep `appsettings.Local.json` on your own machine only.

3.2 Starting and stopping

Start:

```
dotnet run --project C:\Apps\DocLockAdmin
```

The console shows which account each environment connects as:

```
Dev: connecting as IUSA\nuclog
Test: connecting as ISOTEST\nuclog
Prod: connecting as ISODOM\nuclog

DocLockAdmin running at http://localhost:5080  (Ctrl+C to stop)
```

Open **http://localhost:5080** in a browser.

Stop: press **Ctrl+C** in the console. If the port is still in use (an old copy is running):

```
Get-Process DocLockAdmin -ErrorAction SilentlyContinue | Stop-Process -Force
```

After updating the code, stop the site, start it again, and press **Ctrl+F5** in the browser so it loads the new page.

To forget saved Credential Manager passwords and be asked again:

```
dotnet run --project C:\Apps\DocLockAdmin -- --reset-credentials
```

The site only listens on `localhost`; other machines cannot reach it.

3.3 Choosing an environment and database

The sidebar has:

1. **Section switch** – *Document locks* or *User logins*.
2. **Environment** – Dev / Test / Prod. The badge shows the account used.
3. **Client databases** (Document locks only) – every database on every server of that environment that has the CM document tables (`AL.DocumentIndex`). System databases and `alchemy`, `AlchemyStore`, `almain`, `AlterianAuth` are skipped. The list is cached for 30 minutes; use **Refresh** to reload it.

3.4 Document locks

Pick a client database in the sidebar. The page shows summary cards (locked documents, users, server) and two tabs:

Tab	What you can do
Locked documents	Every locked document in the database (document, author, locking user, folder). Filter rows, **Export** to CSV, or **Unlock all**.
By user	User list on the left (with search). Click a user to see the documents they authored, with folder, type, created date and who holds the lock. **Unlock user's documents** clears every lock held by that user.

Steps to unlock one user's documents:

1. Choose the environment and client database.
2. Open **By user**, search for and click the user.
3. Check their document list.
4. Click **Unlock user's documents** and confirm.

Steps to unlock everything in a database:

1. Choose the environment and client database.
2. Open **Locked documents** and check the list (use **Export** to keep a copy).
3. Click **Unlock all**, type the **database name** to confirm.

Unlocking sets `LockingUserID = NULL` – the same as CMAdminTool.

3.5 User logins: searching

Switch the sidebar section to **User logins**.

1. Type the name part of the login, **without** the underscore – e.g. `LAKNOX` finds `LAKNOX_SME`, `LAKNOX_HCM`, … (minimum 3 characters).
2. Click **Search all servers**.

The search runs on **every server** of the selected environment at the same time and fills two tabs:

Tab	Table searched	Where
Application users	`AlterianAuth.AUTH.LoginIndex`	Every server
Alterian Auth	`AL.UserIndex`	Every client database on every server (database names differ per server)

- Results are **remembered per environment**. Switching from Prod to Dev and back does not search again; search again to refresh. The time of the last search is shown under the search box.
- **Nothing is selected** after a search. Tick the rows you want, or use **Select all**.
- Servers or databases that could not be reached are listed, so you know what was not checked.

3.6 Application users tab (AlterianAuth logins)

Shows each matching login from `AlterianAuth.AUTH.LoginIndex`, grouped by server, with status (Enabled / Disabled), `lastlogin`, `created`, `modified`, etc. Password hashes are never shown.

Button	What it does
Select all / Clear selection	Ticks or clears every row.
Export	Downloads the results as CSV (do this before a change if you want a copy).
Reset password	See 3.8.
Enable selected	Sets `LoginIndex.enabled = 1`, then moves the user to the **Users** folder in every client database on that server.
Disable selected	Sets `LoginIndex.enabled = 0`, then moves the user to the **Disabled Users** folder in every client database on that server.

Steps to disable a user:

1. Search for the name.
2. On **Application users**, tick the logins to disable.
3. Click **Disable selected**.
4. Type the **name** (e.g. `LAKNOX`) to confirm.
5. The site searches again and shows a green **Verified** banner if every selected login is now disabled, or a red banner listing what did not change.

Notes:

- The folder move only changes `AL.UserIndex.folderID`. It does **not** change `AL.UserIndex.enabled` – that is done on the Alterian Auth tab.
- A database that does not have the user is skipped. A database without exactly one folder called *Users* / *Disabled Users* (folderType 1) is reported as an error and its folder is not changed.

- `modified` is set to the current date/time on every row that changes.

3.7 Alterian Auth tab (AL.UserIndex)

Shows each matching user from `AL.UserIndex` in every client database, grouped by database, with status, folder, email, title and modified date.

Button	What it does
Select all / Clear selection	Ticks or clears every row.
Enable selected	Sets `AL.UserIndex.enabled = 1` in that database.
Disable selected	Sets `AL.UserIndex.enabled = 0` in that database.

Only the `enabled` and `modified` columns change. The folder and AlterianAuth logins are not touched.

Steps:

1. Search for the name.
2. Open **Alterian Auth**, tick the users.
3. Click **Enable selected** or **Disable selected**, type the **name** to confirm.
4. Check the **Verified** banner.

3.8 Reset password

Port of CMAdminTool's *Reset Password*. On the **Application users** tab:

1. Tick the login(s).
2. Click **Reset password**, type the **name** to confirm.

For each login the site:

1. Clears failed logon attempts (`AUTH.LoginHistory` rows with `failreason > 0`), which unlocks the account.
2. Removes the temporary password from the password history so it can be used again.
3. Sets the password to the temporary password (`ResetPassword` setting) with `AUTH.ChangePasswordNew`.
4. Expires it with `AUTH.ExpirePassword`, so the user **must choose a new password at next logon**.
5. Sets `LoginIndex.modified` to the current date/time.

Give the user the temporary password through your normal secure channel.

3.9 Audit log

Every change is written to `DocLockAdmin\unlock_audit.csv`:

```
Time,RunBy,Environment,Server,Database,Action,Subject,RowsChanged
"2026-11-10T11:46:15","IUSA\akarth as IUSA\nuclog","Dev","VIRMZ...","SME2","DISABLE
USERINDEX","EMILYV_SME",1
```

Actions: `UNLOCK USER DOCS`, `UNLOCK ALL DOCS`, `ENABLE LOGIN`, `DISABLE LOGIN`, `MOVE USER FOLDER`, `ENABLE USERINDEX`, `DISABLE USERINDEX`, `RESET PASSWORD`.

`RunBy` shows both you and the account used on the server.

3.10 Safety checks

- Every change asks for a **typed confirmation** (the name, or the database name for *Unlock all*). The server checks it as well.
- Only logins / users starting with `<NAME>_` can be changed, so a search for `LAKNOX` can never change `LAKNOXX_SME` or someone else.
- Only servers in `Servers.csv` and databases found by discovery can be targeted.
- All SQL uses parameters – names are never pasted into the SQL text.
- Enable/disable only touches rows that are not already in that state, so running it twice is harmless.
- Each environment has its own connection pool, so a Prod connection is never reused for Dev.

3.11 Settings reference

`DocLockAdmin\appsettings.json` → section `DocLock`:

Setting	Default	Meaning
`Url`	`http://localhost:5080`	Address the site listens on
`ServersCsv`	`..\Servers.csv`	Server list (relative to the DocLockAdmin folder)
`ExcludeDatabases`	alchemy, AlchemyStore, almain, AlterianAuth	Never treated as client databases
`TrustServerCertificate`	`true`	Accept the servers' self-signed certificates
`QueryTimeoutSeconds`	`60`	SQL command timeout
`CacheMinutes`	`30`	How long the database list is cached
`AuditLog`	`unlock_audit.csv`	Audit file
`Accounts`	Dev/Test/Prod nuclog accounts	Windows account per environment
`EnvironmentAuth`	`Windows`	`Windows`, or `Sql` to be asked for a SQL login at startup
`AuthDatabase`	`AlterianAuth`	Database holding `AUTH.LoginIndex`
`MinLoginPrefixLength`	`3`	Shortest name you can search for
`EnabledUsersFolder`	`Users`	Folder for enabled users
`DisabledUsersFolder`	`Disabled Users`	Folder for disabled users

`DocLockAdmin\appsettings.Local.json` (not in git): `Passwords` and `ResetPassword`.

3.12 Troubleshooting

Problem	Fix
Login is from an untrusted domain	The environment's password is missing or wrong. Check `appsettings.Local.json` or Credential Manager, or run with `--reset-credentials`.
The certificate chain was issued by an authority that is not trusted	Set `TrustServerCertificate` to `true`.
Page doesn't show the latest changes	Stop all copies of DocLockAdmin, start again, press **Ctrl+F5**.
Build error: file is locked	DocLockAdmin is still running. Stop it (see 3.2).
Expected exactly one user folder…	That database has no folder (or several) named *Users* / *Disabled Users* with folderType 1. Fix the folders in that database.
A server shows *Error* in the results	That server could not be reached. The other servers are still processed.
Database is not a client database	The database list is cached. Click **Refresh** in the sidebar.

---

4. Remove-AppLogins-Interactive.ps1

Command-line version of *Disable* for AlterianAuth logins. Finds every login `LIKE 'NAME[_]%'` in `[AlterianAuth].[AUTH].[LoginIndex]` on every server of one environment and sets `enabled = 0`.

```
# Preview only – shows what would change, changes nothing
.\Remove-AppLogins-Interactive.ps1 -Environment Dev -PreviewOnly

# Disable
.\Remove-AppLogins-Interactive.ps1 -Environment Prod
```

Parameter	Meaning
`-Environment`	`Dev`, `Test` or `Prod` (required)
`-PreviewOnly`	Search and report only
`-UseSqlLogin`	Ask for a SQL login instead of Windows authentication
`-TrustServerCertificate`	Accept self-signed certificates
`-ServerFile`	Server list (default `servers.csv` next to the script)

Steps:

1. Run the script; enter the name when asked (e.g. `LAKNOX`).
2. Review the list. The rows are saved to `before_<env>_<time>.csv` first.
3. Type `DISABLE` to confirm (`DISABLE PROD` in Prod).
4. Results are saved to `disable_log_<env>_<time>.csv` with a status per login: `ENABLED` (now disabled), `ALREADY DISABLED`, `NOT PRESENT`, `ERROR`.

The script does not move folders or change `AL.UserIndex`; use the website for that.

5. Invoke-QueryAllServers.ps1

Runs one **read-only** query on every server of one or more environments and combines the results into one CSV, with `Server`, `Environment` and `Database` columns added.

```
# One query in a given database on every Prod server
.\Invoke-QueryAllServers.ps1 -Environment Prod -Database AlterianAuth -Query "SELECT COUNT(*) AS
n FROM AUTH.LoginIndex"

# Every client database, skipping the system/shared ones
.\Invoke-QueryAllServers.ps1 -Environment Prod -AllDatabases `
    -ExcludeDatabase alchemy,AlchemyStore,almain,AlterianAuth `
    -RequireObject AL.FolderIndex `
    -Query "SELECT * FROM [AL].[FolderIndex] WHERE folderType = 1"

# Query from a file, two environments
.\Invoke-QueryAllServers.ps1 -Environment Dev,Prod -AllDatabases -QueryFile .\query.sql
```

Parameter	Meaning
`-Environment`	One or more of `Dev`, `Test`, `Prod`
`-Query` / `-QueryFile`	The SQL to run (one of them)
`-Database`	Run in this database
`-AllDatabases`	Run in every user database on each server
`-ExcludeDatabase`	Databases to skip with `-AllDatabases`
`-RequireObject`	Only databases that contain this table/view (e.g. `AL.FolderIndex`)
`-UseSqlLogin`	Ask for a SQL login instead of Windows authentication
`-TrustServerCertificate`	Accept self-signed certificates
`-QueryTimeout`	Seconds per query

Only queries starting with `SELECT` or `WITH` are allowed; anything containing `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `EXEC`, etc. is refused. Output: `query_results_<env>_<time>.csv`.

6. What each action changes in SQL

Action	Database	Statement (simplified)
Unlock user's documents	client DB	`UPDATE AL.DocumentIndex SET LockingUserID = NULL WHERE LockingUserID = <user id>`
Unlock all	client DB	`UPDATE AL.DocumentIndex SET LockingUserID = NULL WHERE LockingUserID IS NOT NULL`
Application users → Disable	AlterianAuth	`UPDATE AUTH.LoginIndex SET enabled = 0, modified = GETDATE() WHERE login = @login AND enabled <> 0`
	each client DB on that server	`UPDATE AL.UserIndex SET folderID = <Disabled Users>, modified = GETDATE() WHERE name = @login`
Application users → Enable	AlterianAuth	`UPDATE AUTH.LoginIndex SET enabled = 1, modified = GETDATE() WHERE login = @login AND enabled <> 1`
	each client DB on that server	`UPDATE AL.UserIndex SET folderID = <Users>, modified = GETDATE() WHERE name = @login`
Alterian Auth → Enable / Disable	client DB	`UPDATE AL.UserIndex SET enabled = 1/0, modified = GETDATE() WHERE name = @user AND enabled <> 1/0`
Reset password	AlterianAuth	delete failed `AUTH.LoginHistory` rows; delete temp password from `AUTH.LoginPasswordHistory`; `EXEC AUTH.ChangePasswordNew`; `EXEC AUTH.ExpirePassword`; set `modified`
Script: Remove-AppLogins	AlterianAuth	`UPDATE AUTH.LoginIndex SET enabled = 0 WHERE login = @Login AND enabled <> 0`

Nothing is ever deleted from `LoginIndex` or `UserIndex`.

7. Files that must never be committed

These are listed in `.gitignore`. Do not force-add them:

- `DocLockAdmin/appsettings.Local.json` – passwords
- `before_*.csv`, `disable_log_*.csv`, `delete_log_*.csv`, `query_results_*.csv` – login data
- `DocLockAdmin/unlock_audit.csv` – audit log
- `bin/`, `obj/` – build output